

Testes em Java Spring Boot

repositório: <https://github.com/gabe-pud/testes-cicd>

Dependências importantes

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

JUnit, Mockito, Spring Test, AssertJ, etc.

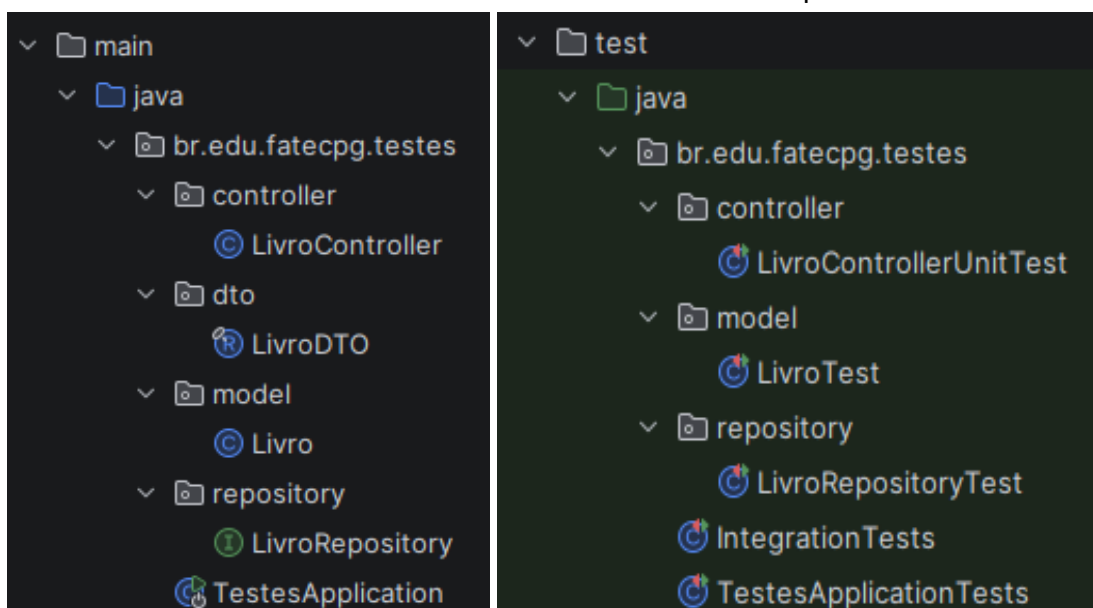
Garanta que o pom.xml contenha spring-boot-starter-test em escopo test.

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

banco em memória usado nos testes de persistência.

Estrutura do projeto

para criar testes em java, devemos criar dentro do diretório “root/src/test/.../”, a mesma estrutura de package do projeto principal “root/src/main/.../”, onde se encontram as classes que queremos testar e então criar classes com o mesmo nome acompanhado de “Test” ou “UnitTest”.



Testes Unitários

Teste simples:

Para criar um teste, dentro de uma classe, criamos métodos sempre void e seguidos da annotation `@Test`, que indica ao java que aquele método deve ser executado como teste.

```
package br.edu.fatecpg.testes.model;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class LivroTest {

    @Test
    void constructorsAndAccessors() {
        Livro l1 = new Livro();
        l1.setTitulo("A");
        l1.setAutor("B");

        assertEquals(expected: "A", l1.getTitulo());
        assertEquals(expected: "B", l1.getAutor());

        Livro l2 = new Livro(titulo: "C");
        assertEquals(expected: "C", l2.getTitulo());
    }
}
```

Neste teste utilizamos apenas Junit para validar getters/setters e construtores.

Definimos as informações em variáveis e então verificamos se obtivemos o resultado esperado com o método `assertEquals()`.

Teste com mocks:

Para testes onde alguma classe busca informações externas, utilizamos a biblioteca Mockito que nos permite “fingir” que as informações estão sendo buscadas.

```
package br.edu.fatecpg.testes.controller;

import ...

@ExtendWith(MockitoExtension.class)
class LivroControllerUnitTest {

    @Mock 2 usages
    LivroRepository repository;

    @InjectMocks 2 usages
    LivroController controller;

    @Test
    void cadastrarSuccess() {
        LivroDTO dto = new LivroDTO(titulo: "Titulo", autor: "Autor");

        Livro saved = new Livro();
        saved.setTitulo("Titulo");
        saved.setAutor("Autor");

        when(repository.save(any(Livro.class))).thenReturn(saved);

        ResponseEntity<?> resp = controller.cadastrar(dto);

        assertEquals(expected: 200, resp.getStatusCode().value());
        assertEquals(LivroDTO.class, resp.getBody());
        LivroDTO body = (LivroDTO) resp.getBody();
        assertEquals(expected: "Titulo", body.titulo());
    }
}
```

Aqui a annotation `@ExtendWith` carrega na classe de teste algumas extensões do mockito que facilitam a definição de mocks, neste caso estamos usando duas annotations delas:

`@Mock` - que cria um objeto que finge ser um objeto da classe que definimos no contexto do teste;

`@InjectMocks` - que injeta os mocks definidos nas dependências de uma classe que será instanciada.

O teste em si segue o mesmo padrão do anterior mas com um passo a mais, agora que temos um mock devemos definir qual vai ser seu comportamento quando chamamos seus métodos, para isso utilizamos os métodos:

- “when()” (com a chamada da função que iremos usar);
- “thenReturn()” (com o resultado que queremos que seja retornado).

Então neste teste, quando o controller tentar executar “repository.save(x)”, o repository irá retornar sempre o “saved” que definimos logo acima.

Teste de Integração:

```
package br.edu.fatecpg.testes;

import ...

@SpringBootTest
class IntegrationTests {

    private MockMvc mockMvc; 2 usages

    @Autowired
    private WebApplicationContext context;

    @Autowired
    private LivroRepository repo;

    @BeforeEach
    void setup() { this.mockMvc = MockMvcBuilders.webAppContextSetup(this.context).build(); }

    @Test
    void postCreatesLivro() throws Exception {
        repo.deleteAll();

        String json = "{\"titulo\":\"ITitle\",\"autor\":\"IAuthor\"}";

        mockMvc.perform(post(uriTemplate: "/livros/cadastrar").contentType("application/json").content(json))
            .andExpect(status().isOk());

        assertEquals(expected: 1, repo.count());
    }
}
```

Aqui utilizamos o `@SpringBootTest` que irá iniciar a aplicação em um contexto em mock utilizando a classe `MockMvc` com a qual podemos testar chamadas de endpoints.

Para preparar o contexto usamos uma função `setup()` com a annotation `@BeforeEach` que irá recriar o contexto antes de cada teste impedindo que estes interfiram nos resultados de outros.

Primeiro criamos uma string json que será o payload da requisição e então usamos os métodos do `mockMvc`:

- “perform()” (com o método http definido para o endpoint e o payload a ser enviado);
- “andExpect()” (com a resposta esperada da requisição).

obs: para método “get” não passamos `contentType` nem `content`.

Como rodar os testes localmente

No Windows PowerShell (na raiz do projeto) use o wrapper Maven:
`./mvnw.cmd test`

Rodar um teste específico (classe):
`./mvnw.cmd -Dtest=LivroRepositoryTest test`

Podemos ver relatórios gerados pelo Surefire em `target/surefire-reports/`.

CI — GitHub Actions (workflow)

```
name: CI

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  build-linux:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Make Maven wrapper executable
        run: chmod +x ./mvnw

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '25'

      - name: Run tests
        run: ./mvnw test
        shell: bash
```

Presente em “`root/.github/workflows/ci.yml`” no repositório, este arquivo configura um workflow que será executado sempre que houver um push ou pull request para a branch main/master.

utilizando uma máquina ubuntu na versão mais recente ele executa as seguintes ações:

Checkout - Usa a action oficial para copiar o código do seu repositório para dentro da máquina virtual;

Make Maven wrapper executable - Executa o comando `chmod +x` para garantir que o script `./mvnw` tenha permissão de execução;

Setup Java - Instala o JDK (Java Development Kit). Aqui, usa a distribuição Temurin na versão 25;

Run tests - executa `./mvnw test` para rodar a suíte de testes unitários do projeto usando o Maven.